Use Case Model: Sudoku Solver System

System Boundary

System: Sudoku Solver Application (C++ program)

Primary Actor: User / System Operator (starts the program to process puzzle files)

Supporting External Resource: File storage containing puzzle1.txt through puzzle5.txt and the location where solution files are written

a. Identify/Discover a Use Case

The core use case is **Solve Sudoku Puzzles**. The application processes five named puzzle files in sequence, validates each file and its starting clues, searches for all valid solutions, displays the results, and writes the results to corresponding solution files.

b. Brief Description of the Use Case

This use case allows a user to run the C++ application to load and validate Sudoku puzzles from puzzle1.txt through puzzle5.txt. Each puzzle must contain exactly 81 valid cell characters (digits 1–9 or underscores for blanks); whitespace between characters is allowed. The system checks that the original clues do not conflict, then uses recursive backtracking to find all solutions. The solver selects an empty cell with the fewest legal candidate numbers to try first. It displays the original puzzle, all solutions found, and the solution count when at least one solution exists. Results are also written to solution1.txt through solution5.txt. The system reports input errors and output-file creation or writing errors.

c. Step-by-Step Flow (Outline) of Events

1 The user runs the Sudoku solver application.

2 The system loops through puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.

3 For each file, the system attempts to open and read the puzzle.

4 The system checks that it can read 81 valid cell characters, allowing whitespace but rejecting invalid characters or extra non-whitespace data. Underscores are stored as 0; digits 1–9 are stored as integers.

5 The system displays the filename and the original puzzle grid.

6 The system validates the starting clues. If clues conflict, the puzzle is treated as having no solution.

7 If the clues are valid, the solver searches recursively using backtracking. It chooses an empty cell with the fewest legal candidates, tries each legal digit, and resets the cell when backtracking.

8 The system displays all discovered solution grids. If no solution exists, it displays “No solution found.”

9 The system writes the solution output to the corresponding solutionN.txt file and reports an error if the output file cannot be created or written.

10 The system proceeds to the next puzzle file and repeats the process.

d. Detailed Description of the Use Case

1. Load and Validate Puzzle Files

User Action: The user starts the application. No further interaction is required while the batch runs.

System Response: The application processes the five expected filenames in order. For each file, it attempts to open an input stream. If the file cannot be opened or read, or if it contains fewer than 81 valid cell characters, an error message is printed and the system moves to the next file. Digits 1–9 represent filled cells and underscores represent blank cells. Whitespace is ignored between cell characters. After reading 81 cells, the system rejects any additional non-whitespace character.

2. Display the Original Puzzle

User Action: None; this step is automatic after a file is loaded successfully.

System Response: The system prints the filename, the “Puzzle:” heading, and the 9×9 grid. Blank cells appear as underscores, and values are separated by spaces. The original puzzle is displayed before solving begins.

3. Validate Clues and Solve Using Backtracking

User Action: None.

System Response: The system checks each given digit against the other values in its row, column, and 3×3 subgrid. If any starting clue conflicts with another, the solver does not begin the search and reports that no solution was found. Otherwise, the recursive solver searches for all solutions. At each step, it selects an empty cell with the fewest legal candidate numbers, which can reduce unnecessary searching. It tries legal digits from 1 through 9, recursively processes the next cell, and resets a cell to blank when backtracking. Every completed grid is stored as a solution.

4. Display and Save Results

User Action: None.

System Response: If no solution is found, the system outputs “No solution found.” If one or more solutions are found, it prints each solved grid and the line “Solutions found: [count]”. The formatted result is built once and sent to the terminal and to the corresponding output file (for example, puzzle1.txt produces solution1.txt). If the output file cannot be opened, the system reports that it could not create the file. If writing fails, it reports a write error. The application then continues to the next input puzzle.

e. Alternative and Error Flows

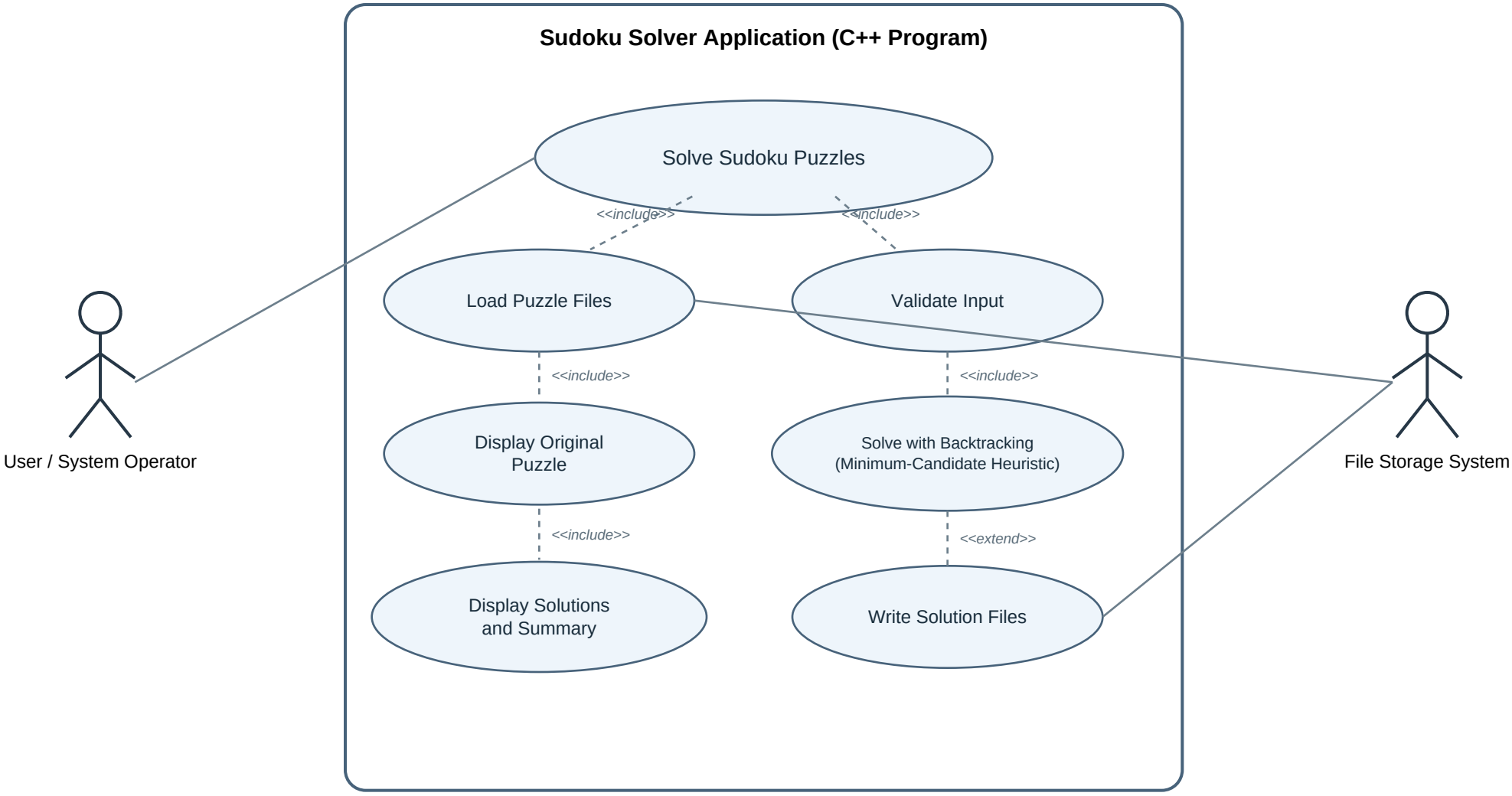
- **Input file missing or unreadable:** The program prints “Could not open or read file.” and continues with the next puzzle.
- **Invalid input format:** If a file contains an invalid character, fewer than 81 cell characters, or extra non-whitespace data, the program rejects the file and continues to the next puzzle.
- **Conflicting starting clues:** The program skips the recursive search and reports “No solution found.”
- **No valid completed grid:** The program reports “No solution found.”
- **Output file cannot be created:** The program prints an error message identifying the output filename.
- **Output write fails:** The program prints an error message identifying the output filename.

Notes on Program Behavior

The program attempts to find and store every valid solution, rather than stopping at the first solution. The minimum-candidate selection strategy changes the order in which empty cells are explored; it does not change the Sudoku rules or the requirement to find all solutions. The program processes the five fixed input filenames and does not request interactive input from the user.

Use Case Diagram – Sudoku Solver System

Updated to reflect the Assignment4.cpp implementation



Main flow

Load and validate each puzzle, display it, search for solutions, then write results.

Error handling Reject malformed input, flag conflicting clues, and report file-write failures.

Solid lines: actor associations • Dashed lines: use-case relationships